Employee Management System (EMS)

A comprehensive full-stack employee management system designed for educational institutions and administrative offices. The system provides role-based access control, employee data management, transfer request processing, and comprehensive reporting features.

🚀 Overview

The Employee Management System is built to handle the complex requirements of managing employees across multiple offices, zones, and districts. It supports hierarchical organizational structures with role-based permissions and provides a complete workflow for employee transfers, promotions, and administrative tasks.

Key Features

- 🔐 **Role-Based Authentication** - Multi-level user roles (CEO, ZEO, School Admin, Staff)
- 👥 **Employee Management** - Complete employee lifecycle management
- 🏢 **Office & Zone Management** - Hierarchical organizational structure
- 📋 **Transfer Management** - End-to-end transfer request workflow
- 📊 **Reporting & Analytics** - Comprehensive reporting system
- 📱 **Responsive Design** - Mobile-friendly interface
- 🔒 **Security** - JWT authentication with password policies
- 📄 **Document Management** - File upload and management
- 📈 **Dashboard Analytics** - Real-time insights and statistics

🏗️ System Architecture

```
graph TB
    subgraph "Frontend - Next.js"
        A[Next.js Application]
        B[React Components]
        C[TanStack Query]
        D[Tailwind CSS]
        E[Axios Client]
    end

    subgraph "Backend - Node.js"
        F[Express Server]
        G[Authentication Middleware]
        H[Controllers]
        I[Services]
        J[Models]
    end

    subgraph "Database"
        K[(MongoDB)]
    end

    subgraph "External Services"
```

```
        L[Cloudinary]
        M[Winston Logger]
    end

    A --> E
    E --> F
    F --> G
    G --> H
    H --> I
    I --> J
    J --> K

    H --> L
    F --> M

    style A fill:#e1f5fe
    style F fill:#f3e5f5
    style K fill:#e8f5e8
```

🗄️ Database Schema

```
erDiagram
    User {
        ObjectId _id PK
        String userName UK
        String role
        String password
        ObjectId office FK
        ObjectId employeeId FK
        Boolean passwordChanged
        ObjectId zoneId FK
        ObjectId districtId FK
    }

    Employee {
        ObjectId _id PK
        String employeeId UK
        String fullName
        String gender
        String maritalStatus
        String staffType
        String presentDesignation
        Date dateOfBirth
        Date dateOfFirstAppointment
        String payScaleAndLevel
        String basicPay
        String email
        String phoneNo
        ObjectId office FK
    }
```

```
    Office {
        ObjectId _id PK
        String officeName
        String officeType
        Boolean isDdo
        String ddoCode UK
        ObjectId parentOffice FK
        String contact
        String address
    }

    Zone {
        ObjectId _id PK
        String name
        ObjectId district FK
        ObjectId[] offices FK
        ObjectId myOffice FK
    }

    District {
        ObjectId _id PK
        String name
        String code UK
    }

    TransferRequest {
        ObjectId _id PK
        ObjectId employee FK
        ObjectId fromOffice FK
        ObjectId toOffice FK
        Date transferDate
        String transferType
        String status
        ObjectId processedBy FK
        Date approvalDate
        String transferOrderNo
    }

    EmployeeSalary {
        ObjectId _id PK
        ObjectId employee FK
        Number basicPay
        Number da
        Number hra
        Number totalSalary
        Date effectiveDate
    }

    Log {
        ObjectId _id PK
        String admin
        String role
        String action
        ObjectId office FK
```

```
        String description
        String ip
        Date timestamp
    }

    User ||--o{ Employee : "manages"
    User }o--|| Office : "belongs to"
    User }o--|| Zone : "manages"
    User }o--|| District : "manages"

    Employee }o--|| Office : "works at"
    Employee ||--o{ TransferRequest : "requests"
    Employee ||--o{ EmployeeSalary : "receives"

    Office }o--|| Office : "parent office"
    Office ||--o{ Zone : "contains"

    Zone }o--|| District : "belongs to"
    Zone ||--o{ Office : "includes"

    TransferRequest }o--|| User : "processed by"

    Log }o--|| Office : "logged at"
    Log }o--|| User : "performed by"
```

🛠️ Technology Stack

Backend

- **Node.js** - Runtime environment
- **Express.js** - Web framework
- **MongoDB** - Database
- **Mongoose** - ODM for MongoDB
- **JWT** - Authentication
- **bcrypt** - Password hashing
- **Winston** - Logging
- **Cloudinary** - File storage
- **Multer** - File upload handling
- **CORS** - Cross-origin requests
- **Helmet** - Security headers

Frontend

- **Next.js 15** - React framework
- **React 19** - UI library
- **TanStack Query** - Data fetching
- **Tailwind CSS** - Styling
- **Axios** - HTTP client
- **React Toastify** - Notifications
- **Recharts** - Data visualization

- **React PDF** - PDF generation
- **React Select** - Enhanced select components
- **Lucide React** - Icons

📦 Installation & Setup

Prerequisites

- Node.js (v18 or higher)
- MongoDB (local or cloud instance)
- Git

Backend Setup

1. **Clone the repository**

```
git clone <repository-url>
cd employee-mgmt-system-main/backend
```

2. **Install dependencies**

```
npm install
```

3. **Environment Configuration** Create a `.env` file in the backend directory:

```
# Database
MONGO_URI=mongodb://localhost:27017/employee_management

# JWT
JWT_SECRET=your_super_secret_jwt_key_here
JWT_EXPIRES_IN=24h

# Cloudinary (for file uploads)
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# Server
PORT=5000
NODE_ENV=development

# CORS Origins
FRONTEND_URL=http://localhost:3000
```

4. **Database Setup**

```
# Start MongoDB service
# For Windows with MongoDB installed
net start MongoDB

# For macOS with Homebrew
brew services start mongodb/brew/mongodb-community

# For Linux
sudo systemctl start mongod
```

5. **Seed Admin User**

```
node seedAdmin.js
```

6. **Start the development server**

```
npm run dev
```

The backend server will start on `http://localhost:5000`

Frontend Setup

1. **Navigate to frontend directory**

```
cd ../frontend
```

2. **Install dependencies**

```
npm install
```

3. **Environment Configuration** Create a `.env.local` file in the frontend directory:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:5000/api
```

4. **Start the development server**

```
npm run dev
```

The frontend application will start on `http://localhost:3000`

🔐 Authentication Flow

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as Backend
    participant DB as Database

    U->>F: Enter credentials
    F->>B: POST /api/auth/login
    B->>DB: Validate user
    DB-->>B: User data
    B->>B: Verify password
    B->>B: Generate JWT token
    B-->>F: JWT token + user data
    F->>F: Store token in localStorage
    F-->>U: Redirect to dashboard

    Note over F,B: Subsequent requests
    F->>B: API request with Bearer token
    B->>B: Verify JWT token
    B-->>F: Protected resource
```

🔑 API Authentication

All API requests (except login/register) require authentication using JWT tokens.

Login Example

```javascript
// Login API call
const response = await axios.post('/api/auth/login', {
  userName: 'admin@example.com',
  password: 'securePassword123',
  loginAs: 'CEO'
});

// Store token
localStorage.setItem('token', response.data.token);

// Configure axios for authenticated requests
axios.defaults.headers.common['Authorization'] = `Bearer ${response.data.token}`;
```

Axios Interceptor Setup

```javascript
// axiosClient.js - Automatic token attachment
axiosClient.interceptors.request.use(
  (config) => {
```

```
    if (typeof window !== "undefined") {
      const token = localStorage.getItem("token");
      if (token) {
        config.headers.Authorization = `Bearer ${token}`;
      }
    }
    return config;
  },
  (error) => Promise.reject(error)
);
```

👥 User Roles & Permissions

Role Hierarchy

```
graph TD
    A[CEO] --> B[ZEO]
    A --> C[District Admin]
    B --> D[School Admin]
    C --> D
    D --> E[Staff]

    style A fill:#ff6b6b
    style B fill:#4ecdc4
    style C fill:#45b7d1
    style D fill:#96ceb4
    style E fill:#feca57
```

Permission Matrix

Feature	CEO	ZEO	School Admin	Staff
Create Users	☑	☑	☑	✕
Manage Employees	☑	☑	☑	☑ (Limited)
Transfer Requests	☑	☑	☑	☑ (Own only)
Approve Transfers	☑	☑	☑	✕
View Reports	☑	☑	☑	☑ (Limited)
Office Management	☑	☑	✕	✕
Zone Management	☑	☑	✕	✕

📋 Core Features

Employee Management

Create Employee

```
const createEmployee = async (employeeData) => {
  try {
    const response = await axiosClient.post('/employees', {
      employeeId: 'EMP001',
      fullName: 'John Doe',
      gender: 'Male',
      maritalStatus: 'Single',
      staffType: 'Teaching',
      presentDesignation: 'Teacher',
      dateOfBirth: '1990-01-01',
      dateOfFirstAppointment: '2020-01-01',
      email: 'john.doe@school.edu',
      phoneNo: '+1234567890',
      office: '507f1f77bcf86cd799439011',
      payScaleAndLevel: 'PB-2',
      basicPay: '35000'
    });

    return response.data;
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Failed to create employee');
  }
};
```

Update Employee

```
const updateEmployee = async (employeeId, updateData) => {
  try {
    const response = await axiosClient.put(`/employees/${employeeId}`,
updateData);
    return response.data;
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Failed to update employee');
  }
};
```

Transfer Management

Create Transfer Request

```
const createTransferRequest = async (transferData) => {
  try {
    const response = await axiosClient.post('/transfers', {
      employee: '507f1f77bcf86cd799439012',
      fromOffice: '507f1f77bcf86cd799439013',
      toOffice: '507f1f77bcf86cd799439014',
      transferDate: '2024-01-01',
```

```
      transferType: 'Transfer',
      transferReason: 'Personal reasons'
    });

    return response.data;
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Failed to create transfer
request');
  }
};
```

Transfer Request Workflow

```
stateDiagram-v2
    [*] --> Pending
    Pending --> UnderReview : Review Started
    UnderReview --> Approved : Accept Request
    UnderReview --> Rejected : Reject Request
    Approved --> Completed : Transfer Executed
    Rejected --> [*]
    Completed --> [*]

    note right of Pending : Initial state when request is created
    note right of UnderReview : Request is being evaluated
    note right of Approved : Request approved, awaiting execution
    note right of Completed : Transfer successfully completed
```

Office & Zone Management

Create Office

```
const createOffice = async (officeData) => {
  try {
    const response = await axiosClient.post('/offices', {
      officeName: 'Central High School',
      officeType: 'Educational',
      isDdo: false,
      ddoCode: 'DDO001',
      parentOffice: '507f1f77bcf86cd799439015',
      contact: '+1234567890',
      address: '123 Education Street, City, State'
    });

    return response.data;
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Failed to create office');
  }
};
```

📊 API Endpoints

Authentication Endpoints

```
POST /api/auth/login              # User login
POST /api/auth/register-zeo       # Register ZEO user
POST /api/auth/register-admin     # Register school admin
POST /api/auth/register-staff     # Register staff user
PUT  /api/auth/update-password    # Update user password
```

Employee Endpoints

```
GET    /api/employees             # Get all employees
GET    /api/employees/:id         # Get employee by ID
POST   /api/employees             # Create new employee
PUT    /api/employees/:id         # Update employee
DELETE /api/employees/:id         # Delete employee
GET    /api/employees/office/:id  # Get employees by office
```

Transfer Endpoints

```
GET    /api/transfers             # Get all transfer requests
GET    /api/transfers/:id         # Get transfer by ID
POST   /api/transfers             # Create transfer request
PUT    /api/transfers/:id         # Update transfer request
PUT    /api/transfers/:id/approve # Approve transfer request
PUT    /api/transfers/:id/reject  # Reject transfer request
```

Office Management Endpoints

```
GET    /api/offices               # Get all offices
GET    /api/offices/:id           # Get office by ID
POST   /api/offices               # Create new office
PUT    /api/offices/:id           # Update office
DELETE /api/offices/:id           # Delete office
```

Zone Management Endpoints

```
GET    /api/zones                 # Get all zones
GET    /api/zones/:id             # Get zone by ID
POST   /api/zones                 # Create new zone
```

```
PUT    /api/zones/:id             # Update zone
DELETE /api/zones/:id             # Delete zone
GET    /api/zones/district/:id    # Get zones by district
```

Reports Endpoints

```
GET    /api/reports/employees     # Employee reports
GET    /api/reports/transfers     # Transfer reports
GET    /api/reports/offices       # Office reports
GET    /api/reports/salary        # Salary reports
```

🔍 Error Handling

Backend Error Response Format

```
// Standard error response
{
  "success": false,
  "message": "Error description",
  "error": "Detailed error information",
  "statusCode": 400
}

// Validation error response
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Email is required"
    },
    {
      "field": "phoneNo",
      "message": "Phone number must be valid"
    }
  ],
  "statusCode": 422
}
```

Frontend Error Handling

```
// Custom error handler service
const handleApiError = (error) => {
  if (error.response?.data?.message) {
    toast.error(error.response.data.message);
```

```
  } else if (error.isHandled) {
    toast.error(error.message);
  } else {
    toast.error('Something went wrong. Please try again later.');
  }

  // Log error for debugging
  console.error('API Error:', error);
};

// Usage in components
try {
  const result = await createEmployee(employeeData);
  toast.success('Employee created successfully!');
} catch (error) {
  handleApiError(error);
}
```

🧪 Testing

Running Tests

```
# Backend tests
cd backend
npm test

# Frontend tests
cd frontend
npm test

# Run specific test file
npm test -- --testPathPattern=employee.test.js

# Run tests with coverage
npm test -- --coverage
```

Test Structure

```
tests/
├── unit/
│   ├── models/
│   ├── controllers/
│   └── services/
├── integration/
│   ├── auth.test.js
│   ├── employee.test.js
│   └── transfer.test.js
└── e2e/
```

```
    ├── login.test.js
    └── employee-crud.test.js
```

🚀 Deployment

Backend Deployment (Node.js)

```
# Build for production
npm run build

# Start production server
npm start

# Using PM2 for process management
npm install -g pm2
pm2 start server.js --name "ems-backend"
pm2 save
pm2 startup
```

Frontend Deployment (Vercel)

```
# Build for production
npm run build

# Deploy to Vercel
npm install -g vercel
vercel

# Or using Vercel CLI with specific settings
vercel --prod
```

Environment Variables for Production

```
# Backend (.env)
NODE_ENV=production
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/ems_production
JWT_SECRET=your_super_secure_production_jwt_secret
CLOUDINARY_CLOUD_NAME=your_production_cloud_name
CLOUDINARY_API_KEY=your_production_api_key
CLOUDINARY_API_SECRET=your_production_api_secret

# Frontend (.env.production)
NEXT_PUBLIC_API_BASE_URL=https://your-backend-domain.com/api
```

📈 Performance Optimization

Database Indexing

```
// Recommended indexes for optimal performance
db.employees.createIndex({ "employeeId": 1 })
db.employees.createIndex({ "office": 1 })
db.employees.createIndex({ "email": 1 })
db.users.createIndex({ "userName": 1 })
db.transferrequests.createIndex({ "employee": 1, "status": 1 })
db.offices.createIndex({ "officeType": 1, "parentOffice": 1 })
```

API Response Caching

```
// Redis caching example
import redis from 'redis';
const client = redis.createClient();

const cacheMiddleware = (duration = 300) => {
  return async (req, res, next) => {
    const key = `cache:${req.originalUrl}`;

    try {
      const cached = await client.get(key);
      if (cached) {
        return res.json(JSON.parse(cached));
      }

      // Store original res.json
      const originalJson = res.json;
      res.json = function(data) {
        // Cache the response
        client.setex(key, duration, JSON.stringify(data));
        return originalJson.call(this, data);
      };

      next();
    } catch (error) {
      next();
    }
  };
};
```

🔒 Security Best Practices

Input Validation

```
import Joi from 'joi';
```

```
const employeeValidationSchema = Joi.object({
  employeeId: Joi.string().required().min(3).max(20),
  fullName: Joi.string().required().min(2).max(100),
  email: Joi.string().email().required(),
  phoneNo: Joi.string().pattern(/^[+]?[1-9]\d{1,14}$/),
  dateOfBirth: Joi.date().max('now').required(),
  office: Joi.string().required()
});

const validateEmployee = (req, res, next) => {
  const { error } = employeeValidationSchema.validate(req.body);
  if (error) {
    return res.status(422).json({
      message: 'Validation failed',
      errors: error.details.map(detail => ({
        field: detail.path[0],
        message: detail.message
      }))
    });
  }
  next();
};
```

Rate Limiting

```
import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP, please try again later.',
  standardHeaders: true,
  legacyHeaders: false,
});

app.use('/api/', limiter);
```

📝 Contributing

Development Workflow

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/amazing-feature`
3. Commit your changes: `git commit -m 'Add some amazing feature'`
4. Push to the branch: `git push origin feature/amazing-feature`
5. Open a Pull Request

Code Style Guidelines

- Use ES6+ features
- Follow camelCase naming convention
- Write meaningful commit messages
- Add JSDoc comments for functions
- Use Prettier for code formatting
- Follow ESLint rules

Pull Request Template

```
## Description
Brief description of changes

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update

## Testing
- [ ] Tests pass locally
- [ ] Added tests for new functionality
- [ ] Manual testing completed

## Checklist
- [ ] Code follows style guidelines
- [ ] Self-review completed
- [ ] Documentation updated
```

📞 Support & Contact

Issues & Bug Reports

- Create an issue on GitHub with detailed description
- Include steps to reproduce
- Provide error logs and screenshots
- Mention your environment details

Feature Requests

- Open a GitHub issue with "Feature Request" label
- Describe the feature and its benefits
- Provide mockups or examples if possible

Documentation

- Wiki: Project Wiki
- API Documentation: API Docs
- Video Tutorials: YouTube Playlist

📋 Changelog

Version 1.0.0 (Current)

- Initial release
- Employee management system
- Transfer request workflow
- Role-based authentication
- Reporting system
- File upload functionality

Planned Features (v1.1.0)

- ☐ Mobile application
- ☐ Advanced reporting
- ☐ Notification system
- ☐ Bulk operations
- ☐ API versioning
- ☐ Performance dashboard

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

🙏 Acknowledgments

- Express.js - Backend framework
- Next.js - Frontend framework
- MongoDB - Database
- Tailwind CSS - CSS framework
- React Query - Data fetching
- Cloudinary - File storage

Employee Management System - Built with 💚 for educational institutions and administrative offices.